IEEE Author Portal
IEEE Transactions on Information Forensics and Security

Sponsoring Society:
IEEE Signal Processing Society | IEEE

Regular Paper

FDXT: Forward and Backward Private Conjunctive Searchable Encryption with Volume-Leakage Suppression

Submission ID e0d4e5aa-19f3-4e75-ab9d-e7c8afef2e81

Submission Version Initial Submission

PDF Generation 18 Aug 2025 09:25:00 EST by Atypon ReX

Authors

Mr. Yuanhong Li

CRediT
Writing - original draft, Conceptualization, Investigation, Methodology, Validation, Formal analysis, Software, Writing - review & editing

Affiliations
- the Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

Prof. Peng Xu
Corresponding Author
Submitting Author

ORCiD
https://orcid.org/0000-0003-4268-4976

CRediT
Funding acquisition, Supervision, Validation, Writing - review & editing, Formal analysis, Writing - original draft

Affiliations
- the Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

Mr. Bochuan Zhang

CRediT
Software, Writing - review & editing, Formal analysis

Affiliations
- the Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

Prof. Wei Wang

CRediT
Funding acquisition, Validation, Writing - review & editing, Formal analysis

Affiliations
- the Cyber-Physical-Social Systems Laboratory, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China

Mr. Yubo Zheng
ORCiD
https://orcid.org/0000-0001-5389-5707
CRediT
Writing - review & editing, Formal analysis, Validation, Methodology

Affiliations
- the Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China

Prof. Kaitai Liang
CRediT
Writing - review & editing, Validation, Formal analysis, Investigation

Affiliations
- the Department of Computing, University of Turku, Finland

Additional Information

EDICS

IF-CRY Applied cryptography / IF-CRY-SEAR Searchable encryption

IF-CRY-SEAR Searchable encryption

Subject Category

APPLIED CRYPTOGRAPHY

Is this manuscript a resubmission of, or related to, a previously rejected manuscript, or a previously reviewed and withdrawn manuscript?

No.

Is this manuscript an extended version of a conference publication (or conference article accepted for publication)?

No.

Is this manuscript related to any other papers of the authors that are either published, accepted for publication, or currently under review, and that are not included among the references cited in the manuscript?

No.

Are there any preprints of the manuscript (i.e. preprints that are identical to the submitted manuscript, except for minor differences) that have been posted on the authors' personal website, employer's website or institutional repository, arXiv.org, TechRxiv.org, or on any not-for-profit preprint server approved by the IEEE?

No.

Are there any other posted preprints that should not be considered to be prior art?

No.

Explain in detail why the contribution of this manuscript is within the scope of the IEEE Transactions on Information Forensics & Security?

This paper contributes to the information security and system applications of searchable encryption by introducing a new conjunctive DSSE scheme, FDXT. It addresses the forward privacy leakage and keyword volume leakage issues that persist in existing conjunctive DSSE constructions, both of which pose significant security risks in practice. Moreover, FDXT achieves performance costs comparable to state-of-the-art designs, enabling practical deployment of searchable encryption in applications.

Why is the contribution significant (What impact will it have)?

Forward privacy prevents correlation between newly generated update queries and prior search queries, and keyword volume suppression eliminates statistical information leakage generated by the searchable encryption. Existing conjunctive DSSE schemes suffer from either forward-privacy or volume-privacy leakage, which makes them vulnerable to injection or leakage-abuse attacks. This paper presents a new conjunctive DSSE scheme that addresses both security issues without compromising efficiency.

What are the three papers in the published literature most closely related to this paper?

1. S. Patranabis et al., "Forward and backward private conjunctive searchable symmetric encryption," NDSS 2021, https://dx.doi.org/10.14722/ndss.2021.23116.
2. C. Zuo et al., "Searchable encryption for conjunctive queries with extended forward and backward privacy," PETS 2025, https://doi.org/10.56553/popets-2025-0024.
3. J. Wu et al., "Practical searchable symmetric encryption for arbitrary boolean query-join in cloud storage," IEEE TIFS, vol. 19, 2024, https://doi.org/10.1109/TIFS.2024.3486002.

What is distinctive/new about the current paper relative to these previously published works?

1. Paper 1 neglected forward privacy in its design. It's update queries for remaining keywords still build relevance to previous search queries.
2. Paper 2 focuses on forward and backward privacy but it incurred high computational overhead and leaked the keyword volume within each conjunctive query.
3. Paper 3 is limited to static settings, while this submission studies dynamic schemes.

In short, our paper achieves both forward and backward privacy, volume-leakage suppression, and efficient performance.

Files for peer review

All files submitted by the author for peer review are listed below. Files that could not be converted to PDF are indicated; reviewers are able to access them online.

Name	Type of File	Size	Page
FDXT__Forward_and_Backward_Private_Conjunctive_Searchable_Encryption_with_Volume_Leakage_Suppression.pdf	Main Document - PDF	2.1 MB	Page 5

FDXT: Forward and Backward Private Conjunctive Searchable Encryption with Volume-Leakage Suppression

Yuanhong Li, Peng Xu, *Senior Member, IEEE,* Bochuan Zhang, Wei Wang, *Member, IEEE,* Yubo Zheng, and Kaitai Liang, *Member, IEEE*

***Abstract*—Dynamic Searchable Symmetric Encryption (DSSE) allows clients to update data and search keywords securely over symmetrically encrypted data on an honest but curious server. Conjunctive DSSE, an attractive type of DSSE with expressive search, enables clients to find data containing multiple keywords simultaneously. However, the recent conjunctive DSSE schemes, like ODXT (in NDSS'21) and SDSSE-CQ (in PETS'25), suffer from either forward-privacy or volume-privacy leakage, which makes them vulnerable to injection or leakage-abuse attack. In this work, we analyze the aforementioned works in depth and design a new conjunctive DSSE scheme named FDXT. For any search query with multiple keywords, FDXT guarantees the forward privacy of all queried keywords. In contrast, ODXT only maintains the forward privacy of the single and first queried keyword. FDXT also avoids volume leakage compared with SDSSE-CQ. Finally, we compared FDXT with ODXT and SDSSE-CQ in terms of performance on the Crime, Wikipedia, and Enron datasets. The experimental results show that FDXT exhibits good performance, which is comparable to ODXT and significantly better than SDSSE-CQ.**

***Index Terms*—Dynamic searchable symmetric encryption, conjunctive query, forward privacy, volume-leakage suppression.**

I. Introduction

Dynamic searchable symmetric encryption (DSSE) [1], an evolutionary extension of searchable symmetric encryption (SSE) [2], allows clients to store their encrypted data on a remote server while dynamically updating these data and issuing keyword queries. DSSE usually assumes that the server is an honest but curious adversary who will handle clients' requests honestly but be curious about clients' data privacy. To maintain data privacy, academics have made numerous efforts to minimize privacy leakage when updating data and searching for keywords while optimizing a balance between security and performance. In practice, DSSE promotes new applications, such as MongoDB's Queryable Encryption [3], Cossack Labs' Acra [4], and IronCore Labs's Cloaked Search [5].

Recently, forward and backward privacy have been widely recognized as a basic security requirement for DSSE to constrain the degree of privacy leakage. Forward privacy [6] can guarantee the independence between the current update operations and historical search queries, such that the adversary cannot use historical search queries to match the subsequently updated data. In practice, a DSSE scheme with forward privacy can mitigate adaptive file injection attack [7], [8]. Backward privacy [9] aims to constrain the relationship between the deleted ciphertexts and the subsequent search queries as much as possible. Since then, academics have proposed various DSSE schemes [10]–[14] to support forward and backward privacy while striving to enhance performance.

However, most existing DSSE constructions that support both forward and backward privacy still limit queries to single-keywords. These schemes lack the ability to support expressive queries and are not always suitable for practical applications. For example, many applications, such as Google Scholar, GitHub, and Microsoft Outlook, need to support multiple-keyword conjunctive searches. Hence, it is valuable and attractive work to design a good conjunctive DSSE scheme. In the context of DSSE, such a conjunctive DSSE scheme can return $\mathbf{DB}(w_1) \wedge ... \wedge \mathbf{DB}(w_n)$ upon receiving a multiple-keyword search query $(w_1, ..., w_n)$, where n denotes the number of keywords of a conjunctive search and $\mathbf{DB}(w_i)$ denotes the set of all document identifiers containing the keyword w_i. In addition, such a conjunctive DSSE should maintain forward and backward privacy, as well as good performance.

A. Challenges

A trivial and heuristic approach to achieve the above work is to apply the single-keyword search of DSSE multiple times, ultimately resulting in a multiple-keyword conjunctive search. Specifically, for a conjunctive search $(w_1, ..., w_n)$, a client can use DSSE to query each keyword independently, retrieve the corresponding file identifiers $\mathbf{DB}(w_1), ..., \mathbf{DB}(w_n)$ from the server, and compute their intersection locally. However, this approach wastes massive computation and communication costs, particularly in the case that the size of $\mathbf{DB}(w_1) \wedge ... \wedge \mathbf{DB}(w_n)$ is much less than the total size of $\mathbf{DB}(w_1), ..., \mathbf{DB}(w_n)$. For example, consider a conjunctive search over a Chinese population dataset with predicates such as "gender = Female" and "city = Wuhan". The trivial approach will retrieve two datasets associated with the above two keywords, respectively, where the "Female" dataset has a size of approximately 690 million. In contrast, the size of the actual required dataset satisfying the above two keywords is only around 5.9 million, which is approximately 1% of the "Female" dataset. Hence, this approach is impractical over large-scale data.

Yuanhong Li, Peng Xu, Bochuan Zhang, and Yubo Zheng are with the Hubei Key Laboratory of Distributed System Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: yuanhongli@hust.edu.cn; xupeng@mail.hust.edu.cn; bochuan_zhang@hust.edu.cn; zhengyubo@hust.edu.cn).

Wei Wang is with the Cyber-Physical-Social Systems Laboratory, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: viviawangwei@hust.edu.cn).

Kaitai Liang is with the Department of Computing, University of Turku, Finland (e-mail: kaitai.liang@utu.fi).

TABLE I

COMPARISONS AMONG CONJUNCTIVE SEARCHABLE SYMMETRIC ENCRYPTION SCHEMES. LET $|D|$, $|W|$, AND $|W_d|$ DENOTE THE TOTAL NUMBER OF DOCUMENTS, TOTAL NUMBER OF DISTINCT KEYWORDS, AND TOTAL NUMBER OF KEYWORDS CONTAINED IN THE UPDATED DOCUMENT, RESPECTIVELY. LET n AND w_1 DENOTE THE NUMBER OF KEYWORDS AND THE LEAST FREQUENTLY UPDATED KEYWORD IN A CONJUNCTIVE SEARCH $q = (w_1, ..., w_n)$, RESPECTIVELY. $|\mathbf{UPD}(w)|$ IS THE TOTAL NUMBER OF DATA UPDATE QUERIES FOR w, AND $|\mathbf{UPD}(q)| = \sum_{i=1}^{n} |\mathbf{UPD}(w_i)|$. $|\mathbf{DB}(w)|$ IS THE NUMBER OF DOCUMENTS MATCHING w, AND $|\mathbf{DB}(q)|$ IS THE NUMBER OF DOCUMENTS MATCHING q. d_{max} DENOTES THE ALLOWED MAXIMUM NUMBER OF DELETION QUERIES. $\mu = \sum_{i=1}^{n} \mu_{w_i}$, WHERE μ_{w_i} DENOTES THE NUMBER OF UPDATED BUT UNSEARCHED ENTRIES FOR w_i. γ AND ξ DENOTE THE PARAMETERS OF BLOOM FILTER. THE FEATURE OF VOLUMES-LEAKAGE SUPPRESSION MEANS THAT THE CONJUNCTIVE KEYWORD VOLUMES OF $(w_2, ..., w_n)$ ARE CONFIDENTIAL.

Scheme	Computation		Communication			Client Storage	FP‡	Volume-leakage suppression
	Search	Update	Search	Update	Round†			
OXT [15]	$O(n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	static	$O(n)$	static	1	$O(1)$	○	✓
HXT [16]	$O(\gamma \cdot n \cdot \lvert\mathbf{DB}(w_1)\rvert)$		$O(\gamma \cdot n \cdot \lvert\mathbf{DB}(w_1)\rvert)$		2	$O(1)$	○	✓
OQXT [21]	$O(n \cdot \lvert\mathbf{DB}(w_1)\rvert)$		$O(n)$		1	$O(1)$	○	✓
SESAME [18]	$O(\xi \cdot \lvert D\rvert)$		$O(\xi)$		1	$O(\xi)$	○	✓
HDXT [23]	$O(\lvert\mathbf{Upd}(w_1)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(\lvert W\rvert/\lvert W_d\rvert)$	$O(\lvert\mathbf{Upd}(w_1)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(\lvert W\rvert/\lvert W_d\rvert)$	2	$O(\lvert W\rvert \cdot \log \lvert D\rvert)$	✗	✓
BDXT [24]	$O(\lvert\mathbf{Upd}(w_1)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(1)$	$O(\lvert\mathbf{Upd}(w_1)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(1)$	2	$O(\lvert W\rvert \cdot \log \lvert D\rvert)$	✗	✓
ODXT [24]	$O(n \cdot \lvert\mathbf{Upd}(w_1)\rvert)$	$O(1)$	$O(n \cdot \lvert\mathbf{Upd}(w_1)\rvert)$	$O(1)$	1	$O(\lvert W\rvert \cdot \log \lvert D\rvert)$	✗	✓
SDSSE-CQ [28]	$O(\sum_{i=1}^{n} \lvert\mathbf{DB}(w_i)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(1)$	$O(\sum_{i=1}^{n} \lvert\mathbf{DB}(w_i)\rvert + n \cdot \lvert\mathbf{DB}(w_1)\rvert)$	$O(1)$	1	$O(\lvert W\rvert \cdot d_{max})$	✓	✗
FDXT (Ours)	$O(\mu + n \cdot \lvert\mathbf{Upd}(w_1)\rvert)$	$O(1)$	$O(\mu + n \cdot \lvert\mathbf{Upd}(w_1)\rvert)$	$O(1)$	1	$O(\lvert W\rvert \cdot \log \lvert D\rvert)$	✓	✓

†The Round indicates the communication round required for client to receive the document identifiers matching the conjunctive search query.
‡The FP indicates the forward privacy for conjunctive DSSE.

To address the above issue, numerous works, as shown in Tab. I, have aimed to design efficient conjunctive SSE schemes. The early works of them focus on static data [15]–[21]. They require clients to encrypt and outsource their data to the server during the setup phase, and then clients cannot update their encrypted data except for issuing search queries. Kamara et al. [22] and Yuan et al. [23] attempt to achieve conjunctive DSSE by introducing an updatable multi-map. However, their approach requires reconstructing the maps during each update operation, which incurs a prohibitive cost for large-scale database systems. Patranabis et al. [24] proposed a conjunctive DSSE framework that offers constant update overhead, sublinear search, and single-round communication. Its core idea, called oblivious dynamic cross tags (ODXT), retrieves conjunctive search results by iteratively matching document entries of the least-frequently-updated keyword w_1[1] with the remaining keywords $(w_2, ..., w_n)$ of the conjunctive query. The ODXT is close to the (quasi-) optimal search and update complexity and has been reproduced multiple times in subsequent implementations [25]–[27]. We will provide a detailed exposition of this approach in Sec. III.

However, it is still challenging to suppress information leakage in conjunctive DSSE. Specifically, in the case of ODXT, its forward privacy is only guaranteed for w_1 in the first phase by the underlying scheme `MITRA` [29]. The update queries for remaining keywords $(w_2, ..., w_n)$ still build relevance to previous search queries. We conduct a detailed analysis of this leakage in Sec. III. This observable state transition leaks the privacy of client's search queries. Zuo et al. [28] also noticed this issue and innovatively attempted to mitigate it by encapsulating cross-tags using `Aura` [30] to enforce independence between cross-term updates and prior search. This approach indeed provides guarantees of forward and backward privacy for conjunctive DSSE, but it also reveals the volume of each keyword. As a form of statistical information, the keyword volume can potentially be exploited by leakage abuse attacks [31]–[34] to recover all queries. We present a detailed analysis of this scheme in Sec. III. Such leakage is unavoidable when directly applying conventional single-keyword forward-private DSSE schemes. Therefore, constructing conjunctive DSSE with *practical*, *minimal leakage*, and *stronger forward and backward privacy* remains a significant challenge.

[1]The keyword with the lowest update frequency refers to the one with the fewest updated entries in a database. Throughout this paper, we set w_1 denote the least frequently updated keyword in a conjunctive search query.

B. Our Contributions

This paper demonstrates the feasibility of designing a practical conjunctive DSSE scheme with stronger security. The core idea is to achieve fast matching between the document set of the least frequent keyword in a search and the remaining keywords, while simultaneously suppressing both forward privacy leakage and keyword volume leakage. We outline our main contributions with a brief overview below.

Leakage Analysis. We analyze existing cross-tag based DSSE constructions and demonstrate their forward-privacy leakage or keyword-volume leakage. Our analysis reveals that the keyword-volume leakage emerges as a direct consequence of mechanisms implemented to suppress the forward-privacy leakage. Building upon existing works [24], [28], we formally present a security model for conjunctive DSSE to define a more precise quantification of both forward-privacy leakage and keyword-volume leakage induced by cross-tag mechanisms. Furthermore, this model is also applicable in cross-tag-based Boolean queries [35], [36] and Join queries [37].

Solution. We present an efficient instantiation of conjunctive DSSE, named Forward Privacy Dynamic Cross-Tags (FDXT), which supports forward privacy and Type-II backward privacy. FDXT is built upon a carefully designed encryption based on search and update counters. This structure preserves forward privacy through deferred cross-tag updates, while its decryption process obscures trapdoors across distinct keywords to suppress the leakage of keyword volumes. Additionally, FDXT avoids redundant processing of repeated ciphertexts, resulting in lower computational costs. The comparative analysis in Tab. I demonstrates the optimal security-performance tradeoff of FDXT.

Evaluations. We conduct a comprehensive evaluation of our scheme and existing works using real-world datasets. Specifically, under equivalent security parameters, we compare FDXT, ODXT [24], and SDSSE-CQ [28] in terms of time cost, communication cost, and storage cost on Crime Reports, Wikipedia, and Enron Email dataset. The evaluation results show that FDXT maintains comparable costs to the state-of-the-art constructions of conjunctive DSSE, while achieving enhanced security. The average search time and communication cost of FDXT on the aforementioned datasets are 30.01 ms and 328.41 KB, respectively. Compared to the existing schemes, FDXT proves to be a competitive solution.

II. Preliminary

In this section, we introduce the notations and background material on cryptographic primitives used throughout this work.

A. Notations

We let $\lambda \in \mathbb{N}$ represent the security parameter. We let $x \xleftarrow{\$} \mathcal{X}$ represent randomly selecting an element x from $\mathcal{X}$. We write $\{0,1\}^d$ to represent binary strings of length $d \in \mathbb{N}$, and write $\{0,1\}^*$ to represent binary strings of arbitrary length. We write $poly(\lambda)$ as a polynomial function in λ, and write $negl(\lambda)$ as a negligible function in λ.

We let $\mathcal{F} = \{f_1, ..., f_{|D|}\}$ denote a collection of D documents, each of which is associated with textual keywords from a given alphabet Λ. We use **DB** to denote the database that consists of several keyword-and-document-identifier pairs, such that pair $(w, id) \in$ **DB** iff the document corresponding to identifier id involves the keyword w, and let $\mathbf{DB}(w)$ denote the set of all document identifiers containing the keyword w. We write W as the set of all keywords appeared in **DB**, write $|W|$ as the count of distinct keywords, and write $|\mathbf{DB}(w)|$ as the count of documents containing the keyword w. We denote $q = \{w_1, ..., w_n\}$ as a conjunctive query, where n represents the number of distinct keywords. We denote the search result as $\mathbf{DB}(q) = \bigcap_{i=1}^{n} \mathbf{DB}(w_i)$. We reiterate that throughout this work, w_1 represents the keyword with the lowest update frequency in q. We denote the number of all keyword-and-document-identifier pairs associated with a keyword as the *keyword volume* of the keyword.

B. Cryptographic Background

This part introduces the syntax and security definitions of the cryptographic primitives employed in this paper.

Pseudorandom functions. Let $F : \{0,1\}^\lambda \times \{0,1\}^\ell \to \{0,1\}^{\ell'}$ be a pseudorandom function (PRF) family. F is a secure PRF family iff for all probabilistic polynomial time (PPT) algorithms $\mathcal{A}$, $|\Pr[\mathcal{A}^{F(K,\cdot)} = 1] - \Pr[\mathcal{A}^{G(\cdot)} = 1]| \leqslant negl(\lambda)$, where $K \xleftarrow{\$} \{0,1\}^\lambda$ and $G : \{0,1\}^\ell \to \{0,1\}^{\ell'}$ is a truly random function.

Decisional Diffie-Hellman Assumption. Assume p is a prime of λ-bit length, $\mathbb{G}$ is a cyclic group with order p, and g is a generator of $\mathbb{G}$. For any PPT algorithm $\mathcal{A}$ and $a, b, c \xleftarrow{\$} \mathbb{Z}_p^*$, the decisional Diffie-Hellman (DDH) assumption is said to hold in $\mathbb{G}$ if it is satisfied that $\left|\Pr[\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr[\mathcal{A}(g, g^a, g^b, g^c) = 1]\right| \leqslant negl(\lambda)$.

Conjunctive DSSE. A conjunctive DSSE scheme consists of three client-server protocols Setup, Update, and Search:

- Setup(λ): The client input parameter λ, obtains a state-set st and a secret key sk, and sends an empty ciphertext database **CDB** to the server.
- Update(sk, st, op, (w, id); **CDB**): The client is able to insert or remove a keyword-and-document-identifier entry (w, id) to **CDB** according to the inputted operation type $op \in \{add, del\}$. The client input the secret key sk and the state-set st, generates and sends a ciphertext of entry $(op, (w, id))$ to the server, and updates its state-set st. Upon receiving the ciphertext from the client, the server updates **CDB**.
- Search(sk, st, q; **CDB**): The client issues an encrypted conjunctive query q with the secret key sk and the state-set st to the server. Subsequently, the server returns the matching result $\mathbf{DB}(q)$ to the client.

A conjunctive DSSE scheme is correct if for all queries q, the Search protocol always returns $\mathbf{DB}(q)$. The formal definition can be found in [1].

A conjunctive DSSE scheme achieves adaptive security if the PPT adversary cannot distinguish between a real game and an ideal game, where the ideal game is parameterized with the leakage functions. In the real game, the Setup, Update, and Search protocols are honestly implemented using real keyword-and-document-identifier entries. In contrast, in the ideal game, a simulator simulates the DSSE responses according to the given leakage functions when receiving the queries from an adversary. The formal security as defined in [9], [38] is as follows.

Definition 1 (Adaptive Security of A Conjunctive DSSE)**.** *A conjunctive DSSE scheme is $\mathcal{L}$-adaptively secure iff for any sufficiently large security parameter $\lambda \in \mathbb{N}$ and PPT adversary $\mathcal{A}$, there is an efficient simulator $\mathcal{S}$ with leakage function $\mathcal{L} = (\mathcal{L}^{Stp}, \mathcal{L}^{Updt}, \mathcal{L}^{Srch})$ such that $\left|\Pr[Real_{\mathcal{A}}^{DSSE}(\lambda) = 1] - \Pr[Ideal_{\mathcal{A},\mathcal{S},\mathcal{L}}^{DSSE}(\lambda) = 1]\right| = negl(\lambda)$, where the games $Real_{\mathcal{A}}^{DSSE}(\lambda)$ and $Ideal_{\mathcal{A},\mathcal{S},\mathcal{L}}^{DSSE}(\lambda)$ are defined as below:*

- *$Real_{\mathcal{A}}^{DSSE}(\lambda)$: The real game runs protocol Setup. Adversary $\mathcal{A}$ adaptively initiates Search and Update queries and records the returned transcripts produced by the real Search and Update protocols. At last, $\mathcal{A}$ returns a bit.*
- *$Ideal_{\mathcal{A},\mathcal{S},\mathcal{L}}^{DSSE}(\lambda)$: All transcripts are replaced by the simulator $\mathcal{S}$. The simulator first returns a simulated **CDB** with the inputted leakage function $\mathcal{L}^{Stp}$. Adversary $\mathcal{A}$ can adaptively initiate the Search and Update queries, same as in the real game. Then $\mathcal{S}$ simulates the corresponding transcripts with the inputted leakage functions $(\mathcal{L}^{Updt}, \mathcal{L}^{Srch})$. Finally, $\mathcal{A}$ returns a bit.*

III. Revisiting Cross-Tag-Based DSSE

In this section, we review the working principles of current cross-tags based conjunctive DSSE schemes and analyze their

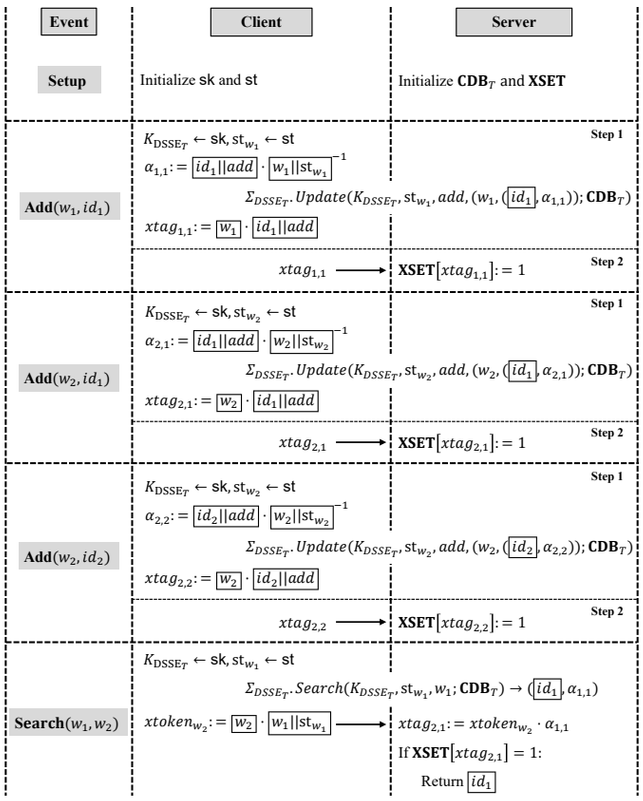


Fig. 1. An example of ODXT. Suppose that the client sequentially adds three entries (w_1, id_1), (w_2, id_1), and (w_2, id_2), and issues a conjunctive query (w_1, w_2) after setupping ODXT. Each black box in this figure denotes a ciphertext.

leakages and the corresponding risks.

The core concept of ODXT designed by Patranabis et al. [24] is illustrated through a concrete example in Fig. 1. When performing opertion **Add**(w_1, id_1), the client first computes a blinded factor $\alpha_{1,1}$ with document identifier id_1, operation label add, keyword w_1, and the local state st_{w_1}. Secondly, the client invokes the Update protocol of a single-keyword DSSE scheme $DSSE_T$ to generate a ciphertext of the encrypted id_1, and stores this ciphertext and blinded factor $\alpha_{1,1}$ in the server's ciphertext database **CDB**$_T$. Thirdly, the client computes a cross-tag $xtag_{1,1}$ with (add, w_1, id_1) and uploads the cross-tag to the server. The server utilizes the cross-tag to update list **XSET**, such that **XSET**$[xtag_{1,1}] = 1$. The procedures to perform operations **Add**(w_2, id_1) and **Add**(w_2, id_2) are similar with that of operation **Add**(w_1, id_1). When issuing a search query **Search**(w_1, w_2), the client identifies which keyword of them has the lowest update frequency according to his local state st. Suppose w_1 has the lowest update frequency without loss of generality. Then, he performs a single keyword search via $DSSE_T$ on w_1 and retrieves the ciphertext of id_1 and blinded factor $\alpha_{1,1}$ from the server's ciphertext database **CDB**$_T$. Then, the client uses the remaining keyword w_2 to compute a cross-token $xtoken_{w_2}$ and sends the cross-token to the server. The server computes the product of $xtoken_{w_2}$ and $\alpha_{1,1}$ to derive the corresponding cross-tag $xtag_{2,1}$. The server returns the ciphertext of the document identifiers that match

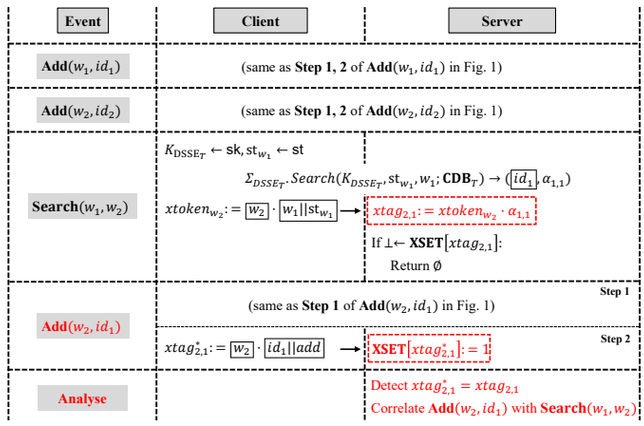


Fig. 2. An example of ODXT's forward-privacy leakage. Suppose that the client sequentially adds two entries (w_1, id_1) and(w_2, id_2), issues a conjunctive query (w_1, w_2), and adds an entry (w_2, id_1). Each black box in this figure denotes a ciphertext.

the conjunctive query **Search**(w_1, w_2), namely the encrypted id_1, to the client if **XSET**$[xtag_{2,1}] = 1$.

Unfortunately, the cross-tag structure of ODXT can leak the forward privacy of the keywords if the client has issued queries to search for these keywords and these keywords are not the ones with the lowest update frequency. We provide a concrete example to illustrate the leakage in Fig. 2. Suppose the client sequentially performs operations **Add**(w_1, id_1) and **Add**(w_2, id_2) and issues a conjunctive query **Search**(w_1, w_2). During the query, the server can record $xtoken_{w_2}$ and $xtag_{2,1}$. Now, suppose the client operates **Add**(w_2, id_1). The server can record $xtag^*_{2,1}$ and infer that the subsequent add operation **Add**(w_2, id_1) is relevant with the previous search query **Search**(w_1, w_2) since $xtag_{2,1} = xtag^*_{2,1}$ holds. Such an inferred relationship violates forward privacy and makes ODXT suffer from the injection attacks [7], [8] in practice.

To avoid the above leakage, SDSSE-CQ [28] provides a method to enhance forward privacy. Fig. 3 shows an example to explain SDSSE-CQ. Suppose the SDSSE-CQ client performs the same operations as the example of ODXT. Fig. 3 uses blue font to highlight the differences between SDSSE-CQ and ODXT. For any add operation **Add**(w, id), SDSSE-CQ and ODXT have the same Step 1; in Step 2, SDSSE-CQ applies another single-keyword scheme $DSSE_X$, instantiated as `Aura` [30], to add the searchable ciphertext of keyword w and the corresponding cross-tag $xtag$ into the encrypted database. When receiving a search query **Search**(w_1, w_2) after completing operations **Add**(w_1, id_1), **Add**(w_2, id_1), and **Add**(w_2, id_2), the SDSSE-CQ server applies $DSSE_X$ to retrieve all the cross-tags associated with keywords w_1 and w_2 and then uses the similar method as ODXT to return the matching and encrypted document identifiers.

However, as shown in Fig. 3, the number of cross-tags per keyword is revealed during the Search of $DSSE_X$. Since each cross-tag represents a keyword-and-document-identifier pair, each cross-tag count directly leaks the number of documents that match its keyword, i.e., the *keyword volume*. Such statistical information leakage is vulnerable to leakage

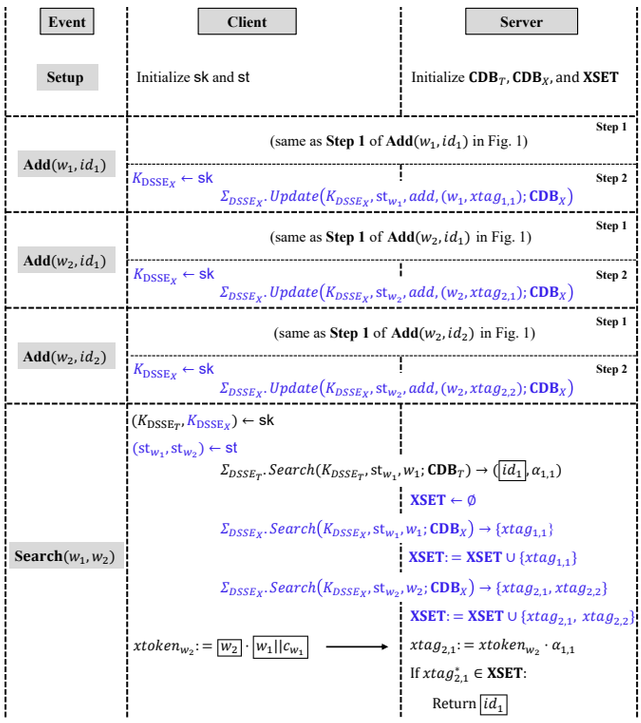


Fig. 3. An example of SDSSE-CQ. Suppose applying SDSSE-CQ to perform the same operations as the above ODXT example does. Each black box in this figure denotes a ciphertext.

abuse attacks [31]–[34], which can exploit the leaked keyword volume, combined with auxiliary knowledge, to recover the keyword. In addition, SDSSE-CQ applies $DSSE_X$ to search every keyword of a conjunctive query independently. Hence, the search complexity of SDSSE-CQ scales linearly with the total number of documents associated with all the queried keywords, which is significantly more than that of ODXT.

IV. Forward Private Conjunctive DSSE with with Volume-Leakage Suppression

To mitigate these limitations, we propose a new conjunctive DSSE construction that simultaneously achieves practical efficiency, forward and type-II backward privacy, and volume-leakage suppression. We first formally present the definitions of forward and backward privacy in the context of conjunctive DSSE, which employs leakage functions to capture the leakages from **XSET**. Then we present FDXT, which uses an encryption based on search and update counters to suppress the leakages discussed earlier.

A. Forward and Backward Privacy

Forward Privacy. The forward privacy [9], [39] requires that the relationships between a newly generated Update queries and arbitrary prior Search queries are hidden. For a conjunctive DSSE, this requirement should be more stringent due to the additional leakage channel introduced by the data structure **XSET**. If forward privacy for **XSET** is neglected, the cross-tags in Update become linkable to those generated during previously issued Search (as evidenced by Fig. 2). Therefore, to achieve forward privacy in conjunctive DSSE, the leakage function of **XSET** must be strictly constrained. We adopt the leakage function design for **XSet** from [28] and formally define forward privacy for conjunctive DSSE.

Definition 2 (Forward Privacy for Conjunctive DSSE)**.** *An $\mathcal{L}$-adaptively secure conjunctive DSSE scheme has forward privacy iff the* Update *leakage function can be written as*

$$\mathcal{L}^{Updt}(op, (w, id)) = \mathcal{L}'((op, id)_T, (op, id)_X),$$

*where $\mathcal{L}'$ is a stateless function, $(op, id)_T$ and $(op, id)_X$ are the leakages of (op, id) for single-keyword DSSE scheme $DSSE_T$ and data structure **XSET**, respectively.*

Backward Privacy. The backward privacy ensures that the historically deleted entries are hidden from the server when running the Search protocol. Before formalizing backward privacy, we recall the necessary functions using notation consistent with [24].

We denote $\mathcal{Q}$ as a query list containing all happened queries, like (u, q) and $(u, op, (w, id))$, where u represents the timestamp of each query. Given a search query $q = \{w_1, ..., w_n\}$, we formally define the following leakage function $\mathsf{TimeDB}(q)$ to return all valid document identifiers matching keywords $w_1, ..., w_n$ together with their respective timestamps u of when they were inserted in the ciphertext database:

$$\mathsf{TimeDB}(q) = \{(\{u_i\}_{i\in[n]}, id \mid (u_i, add, (w_i, id)) \in \mathcal{Q} \quad and \quad \forall u' : (u', del, (w_i, id)) \notin \mathcal{Q}\}.$$

We define function $\mathsf{Upd}(w)$ to capture the leakages of single-keyword DSSE, which outputs the timestamps of each update operation on keyword w:

$$\mathsf{Upd}(w) = \{u \mid \exists (op, id) : (u, op, (w, id)) \in \mathcal{Q}\}.$$

We overload the following function $\mathsf{Upd}(w_1, w_i)$ to capture all cross-term leakages, which outputs the timestamps of each update operation involving document identifiers common to both the w_1 and $w_2, ..., w_n$:

$$\mathsf{Upd}(w_1, w_i)_{i\in[n]} = \{(u_1, u_i)) \mid \exists (op, id) : (u_1, op, (w_1, id)) \in \mathcal{Q} \quad and \quad (u_i, op, (w_i, id)) \in \mathcal{Q}\}.$$

We combine the above leakage functions and define function $\mathsf{Upd}(q)$ for a search query $q = \{w_1, ..., w_n\}$ as:

$$\mathsf{Upd}(q) = \mathsf{Upd}(w_1) \cup (\cup_{i=1}^{n} \mathsf{Upd}(w_1, w_i)).$$

We also define the following function $\mathsf{UpTime}(q)$ to return the timestamps of updated but unsearched operations on $q = \{w_1, ..., w_n\}$:

$$\mathsf{UpTime}(q) = \{u \mid \exists (op, id, \{w_i\}_{i\in n}) : (u, op, (w_i, id)) \in \mathcal{Q} \quad and \quad (u, op, (w_i, id)) \in \mathcal{U}\},$$

where $\mathcal{U}$ records all updated but unsearched entries $(u, op, (w_i, id))$.

With the above leakage functions, we formalize the backward privacy for conjunctive DSSE:

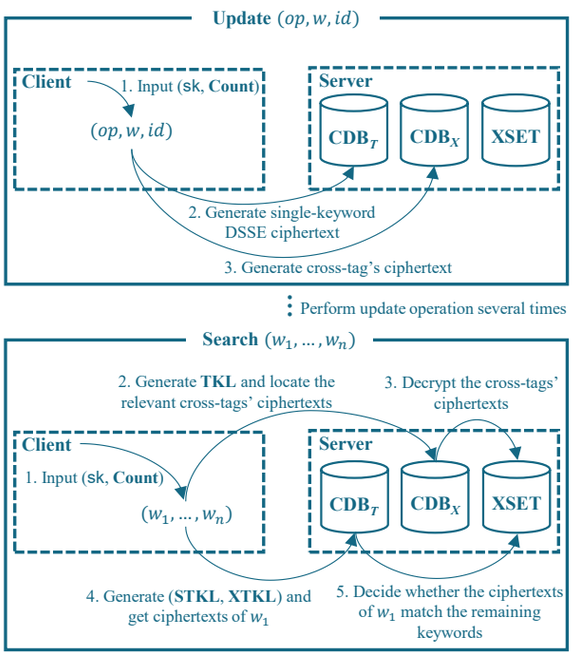


Fig. 4. The FDXT principle. Note that $\mathbf{CDB}_T$ stores the ciphertexts generated by a single-keyword DSSE, $\mathbf{CDB}_X$ stores the unaccessed cross-tags' ciphertexts, and **XSET** determines whether the ciphertexts matching w_1 also satisfy all remaining keywords in a conjunctive query.

Definition 3 (Backward Privacy for Conjunctive DSSE)**.** *An $\mathcal{L}$-adaptively secure conjunctive DSSE scheme has backward privacy iff the* Update *and* Search *leakage functions can be written as:*

$$\mathcal{L}^{Updt}(op, (w, id)) = \mathcal{L}'((op, id)_T, (op, id)_X),$$

$$\mathcal{L}^{Srch}(q) = \mathcal{L}''(\mathsf{TimeDB}(q), \mathsf{Upd}(q), \mathsf{UpTime}(q)),$$

*where $\mathcal{L}'$ and $\mathcal{L}''$ are stateless functions, $(op, id)_T$ and $(op, id)_X$ are the leakages of (op, id) from single-keyword DSSE scheme $DSSE_T$ and data structure **XSET**, respectively.*

The backward privacy definition above restricts timestamp leakage for each $DB(w_i)$, where $w_i \in q$. Furthermore, this definition extends the type-II backward privacy of the single keyword DSSE introduced by [9]. A type-II backward privacy conjunctive DSSE scheme restricts the leakage of a Search query to the access pattern $\mathsf{TimeDB}(q)$ and the partial timestamps of updating $(w_1, ..., w_n)$.

B. FDXT Construction

This section presents FDXT, a practical conjunctive DSSE scheme with forward and backward privacy. FDXT uses an encryption based on the search and update counters to delay updating **XSET** and hides the correlation between the newly generated Update queries and any prior Search queries. Fig. 4 shows the core idea of FDXT. Particularly, when the client issues an Update operation to add/remove an identifier-keyword (id, w) entry to/from the database, FDXT encrypts the corresponding cross-tag with the current values of the search and update counters and uploads the generated ciphertext to the server. This ciphertext consists of the keyword w and its corresponding search and update counters' values. When performing Search, the client generates and uploads search trapdoors for each keyword $w_i \in q$ corresponding to the cross-tags' ciphertexts generated since the last search. The server retrieves and decrypts the corresponding ciphertexts and executes permanent updates by setting $\mathbf{XSET}[xtag] = 1$. At this point, the updated **XSET** can be utilized to perform the conjunctive searches. In short, FDXT achieves efficient conjunctive searches, forward and backward privacy, and keyword volume suppression. The following Algorithms 1, 2, and 3 formally specify the core process of FDXT.

Setup. The FDXT initialization protocol Setup accepts a security parameter λ and initializes two PRFs $\mathbf{F}$ and $\mathbf{F}_p$ and a hash function $\mathbf{H}$. Then, the client randomly samples secret keys $(K_W, K_T, K_X, K_Y, K_Z)$, and initializes three empty maps **Count**, **XSET**, and $\mathbf{CDB} = (\mathbf{CDB}_X, \mathbf{CDB}_T)$. Finally, the client sends **CDB** and **XSET** to the server.

Algorithm 1 Protocol Setup of FDXT.

Setup(λ)

1: Randomly sample secret keys $K_W, K_T, K_X, K_Y, K_Z \xleftarrow{\$} \{0,1\}^\lambda$, and set $\mathsf{sk} \leftarrow (K_T, K_W, K_X, K_Y, K_Z)$
2: Initialize two PRFs $\mathbf{F} : \{0,1\}^* \rightarrow \{0,1\}^{\lambda'}$ and $\mathbf{F}_p : \{0,1\}^* \rightarrow \mathbb{Z}_p^*$, where $\lambda' \leftarrow poly(\lambda)$
3: Choose hash function $\mathbf{H} : \{0,1\}^* \rightarrow \{0,1\}^{\lambda'}$
4: Initialize **Count**, **CDB**, and **XSET** to three maps, and set $\mathbf{CDB} \leftarrow (\mathbf{CDB}_X, \mathbf{CDB}_T)$
5: Send **CDB** and **XSET** to the server

Algorithm 2 Protocol Update of FDXT.

Update(sk, **Count**, op, (w, id); **CDB**)

Client:

1: Parse $(K_W, K_T, K_X, K_Y, K_Z) \leftarrow \mathsf{sk}$, set $(c_w^{srch}, c_w^{updt}, c_w^{max}) \leftarrow \mathbf{Count}[w]$
2: **If** $(c_w^{srch}, c_w^{updt}, c_w^{max}) = (\perp, \perp, \perp)$, **then** initialize $(c_w^{srch}, c_w^{updt}, c_w^{max}) \leftarrow (0, 0, 0)$
3: Set $c_w^{updt} \leftarrow c_w^{updt} + 1$ and $c_w^{max} \leftarrow c_w^{max} + 1$ and update map $\mathbf{Count}[w]$
4: Compute $addr \leftarrow \mathbf{F}(K_W, w||c_w^{max}||0)$ and $val \leftarrow (id, op) \oplus \mathbf{F}(K_W, w||c_w^{max}||1)$
5: Compute $L \leftarrow \mathbf{F}(K_T, w||c_w^{srch}||c_w^{updt}||0)$
6: Compute $T \leftarrow \mathbf{F}(K_T, w||c_w^{srch}||c_w^{updt}||1)$
7: Compute $xtag \leftarrow g^{\mathbf{F}_p(K_X, w) \cdot \mathbf{F}_p(K_Y, id||op)}$ and set $X \leftarrow \mathbf{H}(xtag)$
8: Compute $C \leftarrow X \oplus T$
9: Compute $\alpha \leftarrow \mathbf{F}_p(K_Y, id||op) \cdot \mathbf{F}_p(K_Z, w||c_w^{max})^{-1}$
10: Upload $(addr, val, L, C, \alpha)$ to the server

Server:

1: Set $\mathbf{CDB}_X[L] \leftarrow C$ and $\mathbf{CDB}_T[addr] \leftarrow (val, \alpha)$

Update. To issue an update operation $(op, (w, id))$, the client locally retrieves the current update count c_w^{updt} after the last search, the existing search times c_w^{srch}, and the total update times c_w^{max} of keyword w from **Count**. The client then computes a unique address $addr$, encrypts (id, op) to

Algorithm 3 Protocol Search of FDXT.

Search(sk, **Count**, $q = (w_1, ..., w_n)$; **CDB**)

Client:

Parse $(K_W, K_T, K_X, K_Y, K_Z) \leftarrow$ sk
Select the keyword with the least update times by map **Count** (assume that the identified keyword is w_1 w.l.o.g)
Initialize lists **TKL**, **STKL**, and **XTKL**
for $i = 1$ to n **do**
  Retrieve $(c_{w_i}^{srch}, c_{w_i}^{updt}, c_{w_i}^{max}) \leftarrow$ **Count**$[w_i]$
  Abort if **Count**$[w_i]$ is $(\perp, \perp, \perp)$
  for $l = c_{w_i}^{updt}$ to 1 **do**
    Compute $L_l \leftarrow \mathbf{F}(K_T, w_i || c_{w_i}^{srch} || l || 0)$
    Compute $T_l \leftarrow \mathbf{F}(K_T, w_i || c_{w_i}^{srch} || l || 1)$
    Set **TKL** $\leftarrow$ **TKL** $\cup \{(L_l, T_l)\}$ and shuffle **TKL**
  end for
end for
for $j = 1$ to $c_{w_1}^{max}$ **do**
  Compute $addr_j \leftarrow \mathbf{F}(K_W, w_1 || j || 0)$
  Set **STKL**$[j] \leftarrow addr_j$
  for $k = 2$ to n **do**
    Set $xtk_{j,k} \leftarrow g^{F_p(K_X, w_k) \cdot F_p(K_Z, w_1 || j)}$
    Let **XTKL**$[j][k] \leftarrow xtk_{j,k}$
  end for
end for
Upload (**TKL**, **STKL**, **XTKL**) to the server

Server:

Initialize an empty set **RES**
for $i = 1$ to $|$**TKL**$|$ **do**
  Parse $(L_i, T_i) \leftarrow$ **TKL**$[i]$
  Retrieve $C_i \leftarrow$ **CDB**$_X[L_i]$
  Compute $X_i \leftarrow C_i \oplus T_i$
  Set **XSET**$[X_i] \leftarrow 1$
end for
for $j = 1$ to $|$**STKL**$|$ **do**
  Set $cnt_j \leftarrow 1$
  Retrieve $(val_j, \alpha_j) \leftarrow$ **CDB**$_T$[**STKL**$[j]$]
  for $k = 2$ to n **do**
    Retrieve $xtk_{j,k} \leftarrow$ **XTKL**$[j][k]$
    Compute $X_{j,k} \leftarrow \mathbf{H}((xtk_{j,k})^{\alpha_j})$
    If **XSET**$[X_{j,k}] = 1$, **then** let $cnt_j \leftarrow cnt_j + 1$
  end for
  Insert (val_j, cnt_j) to **RES**$[j]$
end for
Return **RES** to the client

Client:

Initialize an empty set **IDL**
for $j = 1$ to $|$**RES**$|$ **do**
  Parse $(val_j, cnt_j) \leftarrow$ **RES**$[j]$
  Decrypt $(id_j || op_j) \leftarrow val_j \oplus \mathbf{F}(K_W, w_1 || j || 1)$
  If $op_j = add$ **and** $cnt_j = n$, **then** add $\{id_j\}$ to **IDL**
  If $op_j = del$ **and** $cnt_j > 0$, **then** delete $\{id_j\}$ from **IDL**
end for
for $i = 1$ to n **do**
  Let $c_{w_k}^{srch} \leftarrow c_{w_i}^{srch} + 1$ and $c_{w_i}^{updt} \leftarrow 0$ and update these modifications to **Count**$[w_i]$
end for
Output **IDL**

a ciphertext val, and sends $(addr, val)$ to the server. The server updates the ciphertext database **CDB**$_T$ upon receiving the tuple $(addr, val)$. In addition, the client uses secret key K_T and the search and update counters c_w^{srch} and c_w^{updt} of w to generate an address L and a trapdoor T. The trapdoor T is used to encrypt the cross-tag $xtag$, and the generated ciphertext is stored at the address L of **CDB**$_X$.

Search. For a search query $q = (w_1, ..., w_n)$, the client retrieves secret keys $(K_W, K_T, K_X, K_Y, K_Z)$ firstly and counters $(c_{w_i}^{srch}, c_{w_i}^{updt}, c_{w_i}^{max})$ for $w_i \in q$. Subsequently, the client uses $c_{w_i}^{srch}$ and $c_{w_i}^{updt}$ to generate trapdoors for each keyword $w_i \in q$. The server can utilize these generated trapdoors to retrieve and decrypt the cross-tags' ciphertexts. To obscure the *keyword volumes* of keywords $(w_2, ..., w_n)$, these trapdoors are stored in a shuffled list **TKL**. Finally, the client uses c_w^{max} to identify the keyword w_1 with the fewest updates and generates a search token **STKL** of keyword w_1 and a search token **XTKL** of keywords $(w_2, ..., w_n)$. Upon receiving **TKL**, **STKL**, and **XTKL**, the server first locates and decrypts each cross-tag's ciphertext to obtain cross-tag $xtag$, which will be used to update **XSET**. Then, the server utilizes the search token **STKL** to retrieve the tuples (val, α) associated with the keyword of the fewest updates and obliviously computes the cross-tags relevant to the current search using the remaining keywords' search token **XTKL** and α. The server uses the cross-tags as addresses to access **XSET** and utilizes the returned results to determine whether each update entry of keyword w_1 matches the remaining keywords. Finally, the client decrypts the conjunctive search results with secret key K_W and updates the corresponding local counters.

An example of FDXT. We further explain FDXT using an example as shown in Fig. 5. First, in **Add**(w_1, id_1), the client retrieves the update counter $c_{w_1,0}^{updt}$ (which has the initial value zero and records the update count of keyword w_1 from its last search) and the search counter $c_{w_1}^{srch}$ (which records the total search times of keyword w_1). The client sets $c_{w_1,1}^{updt} = c_{w_1,0}^{updt} + 1$ and takes w_1, $c_{w_1,1}$, $c_{w_1}^{srch}$, and a pair of bits as inputs to compute two values $T_{1,1}$ and $L_{1,1}$. The client uses $T_{1,1}$ to encrypt the cross-tag of (w_1, id_1, add) and generate a ciphertext $C_{1,1}$. The client sends $(L_{1,1}, C_{1,1})$ to the server for updating **CDB**$_X$ and updates his local state. FDXT also leverages a single-keyword DSSE scheme to upload the blinding factor $\alpha_{1,1}$ and document identity's ciphertext to **CDB**$_T$ as ODXT does. The operations **Add**(w_2, id_1) and **Add**(w_2, id_2) follow the similar procedures as the above. When issuing a search query **Search**(w_1, w_2), the client retrieves the values of the update and search counters of (w_1, w_2) from the local state **Count**. Then, the client iteratively computes trapdoors (L, T) using

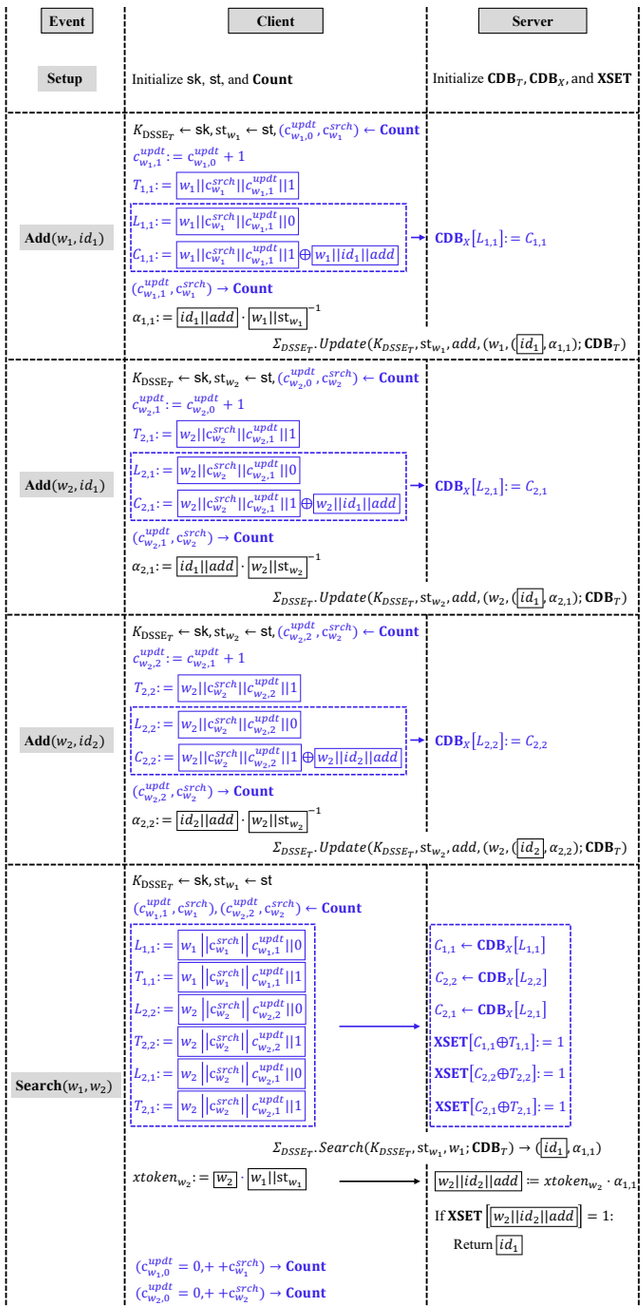


Fig. 5. An example of FDXT. Suppose applying FDXT to perform the same operations as the ODXT example does. Each black box in this figure denotes a ciphertext.

the same method as in the update phase until all keywords' update counters decrement to zero and sends the resulting trapdoors $((L_{1,1}, T_{1,1}), ..., (L_{2,1}, T_{2,1}))$ to the server. Using these trapdoors, the server locates and decrypts the unaccessed cross-tags associated with the current conjunctive query and uses these cross-tags to update **XSET**. Finally, the server uses the updated **XSET** to evaluate whether the ciphertexts of keyword w_1 match the remaining keyword w_2 and returns the correct conjunctive search result. After completing the search, the client resets $c_{w_1}^{updt}$ and $c_{w_2}^{updt}$ to zero while incrementing $c_{w_1}^{srch}$ and $c_{w_2}^{srch}$ by 1.

C. Security Proof

Theorem 1. *Assume functions $\boldsymbol{F}$ and $\boldsymbol{F}_p$ are secure PRFs, function $\boldsymbol{H}$ is a random oracle, and the DDH assumption holds. The FDXT is $\mathcal{L}$-adaptively forward and Type-II backward private, where $\mathcal{L} = (\mathcal{L}^{Updt}, \mathcal{L}^{Srch})$, $\mathcal{L}^{Updt}(op, w, id) = \perp$, and $\mathcal{L}^{Srch}(w) = (\mathsf{TimeDB}(q), \mathsf{Upd}(q), \mathsf{UpTime}(q))$.*

Proof Sketch. The security proof employs a sequence of games similar to [30], [40]. We progressively replace the real cryptographic tools adopted by FDXT with the corresponding simulators, thereby constructing the FDXT simulator $\mathcal{S}$. We prove formally that the views of the adversary $\mathcal{A}$, defined in Definition 1, in any two consecutive games, are computationally indistinguishable.

Proof of Theorem 1: We analyze the computational indistinguishability between the real FDXT and the constructed simulator $\mathcal{S}_{FDXT}$ by following games.

Game G_0: This game is identical to the real FDXT security experiment $Real_{\mathcal{A}}^{\mathrm{FDXT}}(\lambda)$. Therefore, there exists $\mathbb{P}[Real_{\mathcal{A}}^{\mathrm{FDXT}}(\lambda) = 1] = \mathbb{P}[\mathsf{G}_0 = 1]$.

Game G_1: This game models the PRF $\mathbf{F}$ as a random oracle. Given a fresh input, oracle $\mathbf{F}$ will sample and return a λ'-bit string. If the input has occurred, oracle $\mathbf{F}$ returns the same string chosen in the previous issue of the oracle. We note that the observation of $\mathcal{A}$ in G_1 is computationally indistinguishable from it's observation in G_0 if $\mathbf{F}$ is a secure PRF. Therefore, assuming the existence of a PPT reduction algorithm $\mathcal{B}_1$, we have $|\mathbb{P}[\mathsf{G}_1 = 1] - \mathbb{P}[\mathsf{G}_0 = 1]| \leqslant 2\mathsf{Adv}_{\mathbf{F},\omega,\mathcal{B}_1}^{\mathrm{PRF}}(\lambda)$, where ω is linearly related to the sum of *keyword volumes* in a conjunctive query.

Game G_2: This game replicates G_1 but modifies the PRF $\mathbf{F}_p$ to a new random oracle of range domain $\mathbb{Z}_p^*$. Oracle $\mathbf{F}_p$ has a similar working principle to the above oracle $\mathbf{F}$. Similarly, the observation of $\mathcal{A}$ in G_2 is computationally indistinguishable from the observation in G_1 if $\mathbf{F}_p$ is a secure PRF. Assuming the existence of a PPT reduction algorithm $\mathcal{B}_2$, we have $|\mathbb{P}[\mathsf{G}_2 = 1] - \mathbb{P}[\mathsf{G}_1 = 1]| \leqslant 2\mathsf{Adv}_{\mathbf{F}_p,\varepsilon,\mathcal{B}_2}^{\mathrm{PRF}}(\lambda)$, where ε is linearly related to the number of keywords n in a conjunctive search.

Game G_3: This game replicates G_2 but modifies the method used to generate search tokens, like xtk, during a conjunctive search query. Specifically, for search query $q = (w_1, ..., w_n)$, game G_3 derives all the update operations, like $(op_j, (id_j, w_1))$, involving w_1 by retrieving the historical update queries. Then, game G_3 computes α_j, the corresponding cross-tag $xtag_{j,k}$, and the search token $xtk_{j,k} = {xtag_{j,k}}^{1/\alpha_j}$ with keyword $w_k \in q$. Since the above modification does not influence the distribution of $xtk_{j,k}$, we have $\mathbb{P}[\mathsf{G}_3 = 1] = \mathbb{P}[\mathsf{G}_2 = 1]$.

Game G_4: This game modifies the method to evaluate the blinding factor α in an update operation compared with game G_3. For a fresh update operation, game G_4 sets the blinding factor α to be a randomly sampled element from the domain $\mathbb{Z}_p^*$. For a repeated update operation, game G_4 sets the blinding factor α to be the same element sampled in the previous. Since the computation process for α in game G_3 is a product of a random element and the inverse of another random element

both from $\mathbb{Z}_p^*$, game G_4 does not alter the distribution of α in the observation of the adversary $\mathcal{A}$. Therefore, there exists $\mathbb{P}[\mathsf{G}_4 = 1] = \mathbb{P}[\mathsf{G}_3 = 1]$.

Game G_5: This game is identical to G_4 but modifies the way of evaluating cross-tag $xtag$ in an update operation. Specifically, for a fresh update operation, game G_5 lets g is the generator of $\mathcal{G}$, samples a random element β from $\mathbb{Z}_p^*$, and lets $xtag = g^\beta$. For the repeated update operation, game G_5 sets the cross-tag $xtag$ to be the same value chosen in the previous. Hence, in the observation of $\mathcal{A}$, G_5 is computationally indistinguishable from G_4 if the DDH assumption holds. Assuming the existence of a PPT reduction algorithm $\mathcal{B}_3$, we have $|\mathbb{P}[\mathsf{G}_5 = 1] - \mathbb{P}[\mathsf{G}_5 = 1]| \leqslant \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{B}_3}(\lambda)$.

Game G_6: This game modifies the construction of the transcript for each update operation. Namely, for each update operation, game G_6 replaces the corresponding evaluation processes $\mathbf{F}(K_W||w||c_w^{max}||b)$ and $\mathbf{F}(K_T||w||c_w^{srch}||c_w^{updt}||b)$ with $\mathbf{F}(K_W||u)$ and $\mathbf{F}(K_T||u)$, respectively, where $b \in \{0, 1\}$, and u is the timestamp of the corresponding update operation to be searched. In the same way, for the transcript of each conjunctive search operation, game G_6 replaces $\mathbf{F}(K_W||w||j||1)$ and $\mathbf{F}(K_T||w||c_w^{srch}||i||b)$ with $\mathbf{F}(K_W||u)$ and $\mathbf{F}(K_T||u)$, respectively, where $b \in \{0, 1\}$, and u is the timestamp to execute the conjunctive search operation. Since to the existence of monotonically increasing counter variables, the above modified evaluation processes do not appear to have the same input at two different timestamps. In addition, game G_1 has modeled the PRF $\mathbf{F}$ as a random oracle of range domain λ'-bit strings. Hence, games G_6 and G_5 have the identical distribution in the view of the adversary $\mathcal{A}$. We have $\mathbb{P}[\mathsf{G}_6 = 1] = \mathbb{P}[\mathsf{G}_5 = 1]$.

Simulator: Game G_6 is a simulator to simulate the ideal game of FDXT. It just takes the leakage functions as input to simulate all update/conjunctive queries from the adversary $\mathcal{A}$. For any update query, game G_6 takes the timestamp of the query as input and properly simulates the update process. For any search query $q = (w_1, ..., w_n)$, game G_6 can properly simulate Search by accessing the intra-query and inter-query leakages. The intra-query leakage includes the count of the update operations corresponding to w_1, the timestamps of these update operations, and the count of the update operations $(op_j, (id_j, w_i))$, where (op_j, id_j) is an update operation of w_1 and $w_i \in \{w_2, ..., w_n\}$. The inter-query leakage enables game G_6 to determine whether multiple search queries have the same w_1 and whether they have the same w_i in the previously queried document identifiers. The above two types of leakages are encapsulated by the leakage function $\mathcal{L} = (\mathcal{L}^{Updt}, \mathcal{L}^{Srch})$, where $\mathcal{L}^{Updt}(op, w, id) = \perp$ and $\mathcal{L}^{Srch}(w) = (\mathsf{TimeDB}(q), \mathsf{Upd}(q), \mathsf{UpTime}(q))$. Consequently, we construct an effective simulator, namely game G_6, using the leakage function $\mathcal{L} = (\mathcal{L}^{Updt}, \mathcal{L}^{Srch})$, and have $\mathbb{P}[\mathsf{G}_6 = 1] = \mathbb{P}[Ideal^{\mathrm{FDXT}}_{\mathcal{A},\mathcal{S},\mathcal{L}}(\lambda) = 1]$.

Finally, according to the above games and their indistinguishability, the advantage of the adversary $\mathcal{A}$ against FDXT is bounded by

$$\left|\mathbb{P}[Real^{\mathrm{FDXT}}_{\mathcal{A}}(\lambda) = 1] - \mathbb{P}[Ideal^{\mathrm{FDXT}}_{\mathcal{A},\mathcal{S},\mathcal{L}}(\lambda)\right|$$

$$\leqslant 2\mathsf{Adv}^{\mathrm{PRF}}_{\mathbf{F},\omega,\mathcal{B}_1}(\lambda) + 2\mathsf{Adv}^{\mathrm{PRF}}_{\mathbf{F}_p,\varepsilon,\mathcal{B}_2}(\lambda) + \mathsf{Adv}^{\mathrm{DDH}}_{\mathcal{B}_3}(\lambda).$$

V. Implementations

This section experimentally compares FDXT, ODXT [24], and SDSSE-CQ [28] with the same security parameters to demonstrate that FDXT achieves (quasi-)optimal performance while providing enhanced security.

A. Experimental Setup

Our evaluation platform is AMD Ryzen™ 9 9950X CPU @ 4.3GHz, 192GB memory, ZHIATAI Ti7100 4TB NVMe SSD, and Ubuntu 22.04.1 LTS. We code the above three schemes in the Go (version 1.23) language with the Go standard cryptographic libraries[2], which provides AES-256, HMAC-SHA256, and SHA256. We employ AES-256 in CBC mode, HMAC-SHA256, and SHA-256 to realize symmetric encryption, PRFs, and hash functions, respectively, use the Go bloom filter library[3] to realize the bloom filter, and adopt the Type A parings in Gopbc library[4] to implement the group operations. We store all ciphertexts **CDB** in MySQL and employ a bloom filter to construct **XSET**. We extract the following three real datasets to test FDXT, ODXT, and SDSSE-CQ.

Chicago Crimes Reports [41]. The Crime dataset contains 7,989,987 pairs drawn from the incident reports dated 1 Jan 1999 to 2 Apr 2024, where street names serve as searchable keywords mapped to IUCR codes. Finally, there are 63,659 distinct keywords, and the highest keyword volume is 16,644.

Wikipedia [42]. We extract keywords from the processed data using Wikipedia Extractor [43] and the Python NLTK package [44]. Each Wikipedia article serves as a single document, and let the article's number be the corresponding document's identifier. Finally, there are 4,565,948 pairs of keyword-document-identifiers, including 10,000 distinct keywords, and the highest keyword volume is 9,738.

Enron [45]. The Enron email dataset contains 5,190,199 keyword-email pairs index entries generated through NLP preprocessing using NLTK with Porter stemming [46]. Finally, there are 16,241 distinct keywords, and the highest keyword volume is 26,946.

Moreover, we configure the document deletion rate to be 10% and the rate of the updated-but-unsearched documents to be 10% for all the above compared schemes. We employ the bloom filter of thirteen hash functions and the false positive rate $p = 10^{-4}$ to realize SDSSE-CQ.

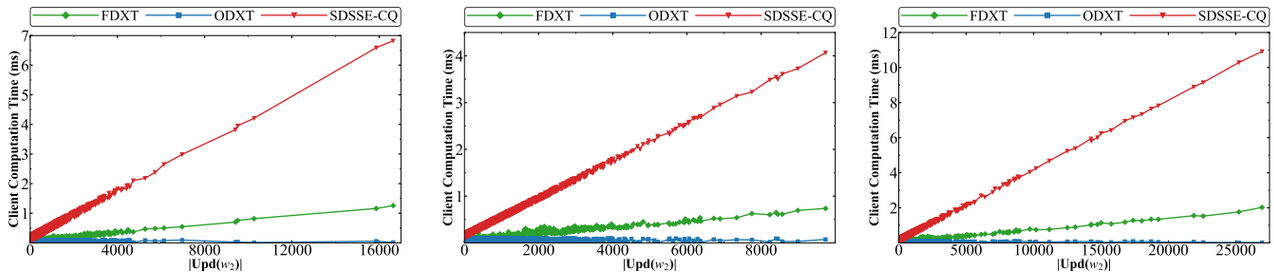
B. Comparison Results

TABLE II
Comparison of Average Update Time Cost (ms/ind).

	FDXT	ODXT	SDSSQ-CQ
$op = add$	8.47	4.33	9.13
$op = del$	8.47	4.33	8.79

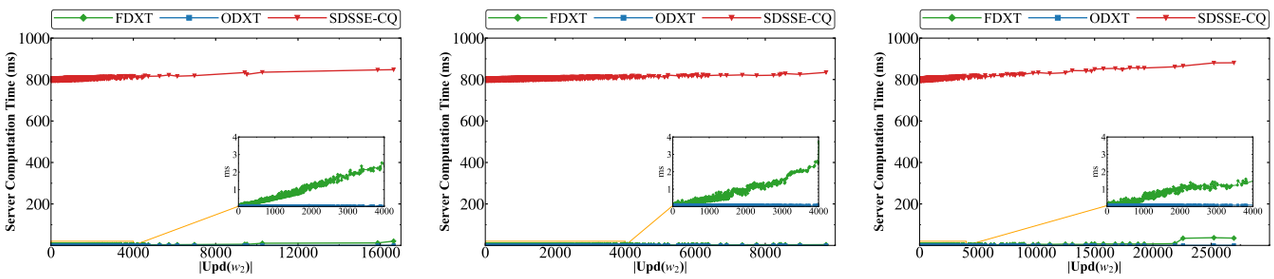
We conducted a comprehensive comparison of the computational, communication, and storage costs of FDXT, ODXT,

[2]Go standard cryptographic libraries: https://pkg.go.dev/crypto
[3]Go bloom filter library: https://github.com/bits-and-blooms/bloom
[4]Gopbc library: https://github.com/Nik-U/pbc



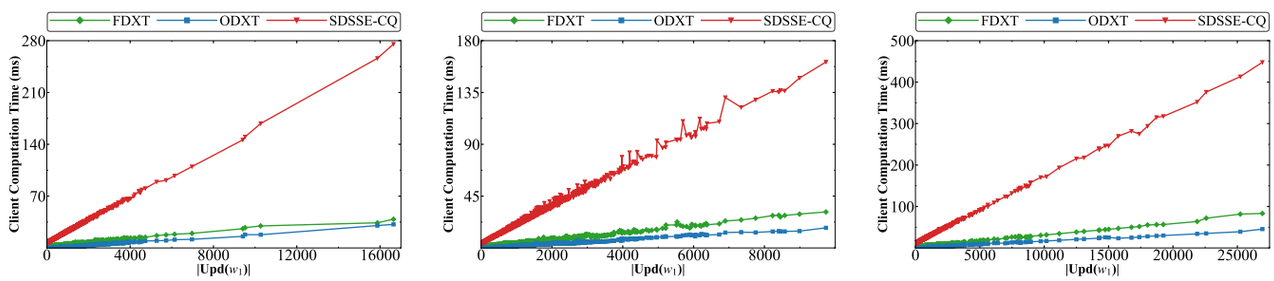
(a) Crime (b) Wikipedia (c) Enron

Fig. 6. Comparative analysis of the client's Search time overheads of FDXT, ODXT, and SDSSE-CQ with a fixed $|\mathbf{Upd}(w_1)| = 10$.



(a) Crime (b) Wikipedia (c) Enron

Fig. 7. Comparative analysis of the server's Search time overheads of FDXT, ODXT, and SDSSE-CQ with a fixed $|\mathbf{Upd}(w_1)| = 10$.



(a) Crime (b) Wikipedia (c) Enron

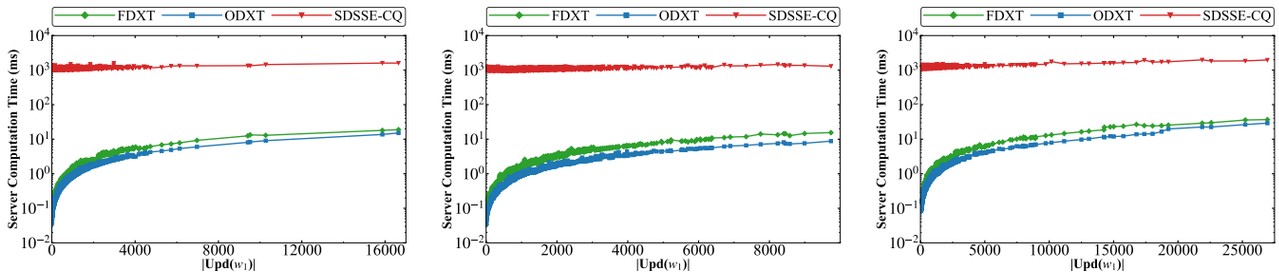
Fig. 8. Comparative analysis of the client's Search time overheads of FDXT, ODXT, and SDSSE-CQ with $|\mathbf{Upd}(w_2)|$ fixed to the highest keyword volume.

and SDSSE-CQ using the above datasets. To provide a clearer illustration of performance trends, we report the specific costs of these schemes across a range of keywords, covering nearly the entire *keyword volume* distribution for each dataset.

Update Time Cost. Tab. II presents the Update performance evaluation of all compared schemes. We observe that FDXT outperforms SDSSE-CQ, achieving approximately 9.5% reduction in time cost. This advantage stems from the fact that SDSSE-CQ requires invoking two instances of AURA to generate revocable ciphertexts, which involves a multi-puncturable PRF based on the GGM-tree construction. Compared to ODXT, FDXT only requires a slight extra cost ($\leqslant 4.14ms$) to support an update query on average. The extra cost is due to FDXT employing an additional encryption to suppress forward privacy leakage of **XSET**. Nevertheless, the cost of FDXT remains lightweight in practice.

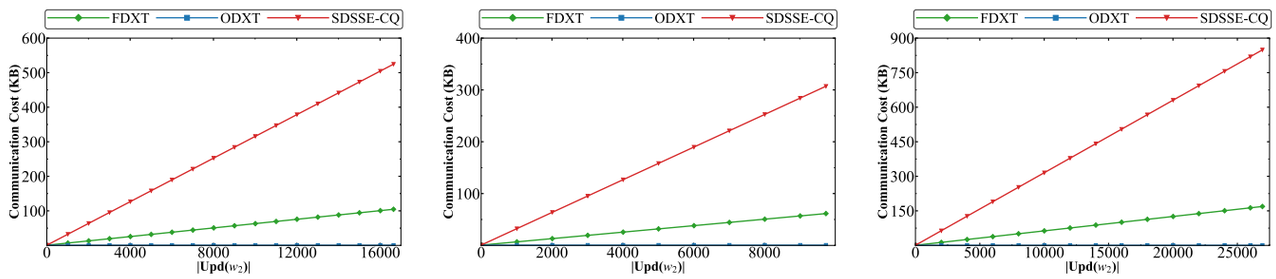
Search Time Cost. We present the search time overheads of client and server for evaluated schemes in response to the search query $q = (w_1, w_2)$. We analyze the impact of w_1 and w_2 on search time in the following two cases. In the first case, we select and fix w_1 such that $|\mathbf{Upd}(w_1)| = 10$, while varying the *keyword volume* of w_2 from 10 to the highest keyword volume in each dataset. In the second case, we fix w_2 as the keyword with the highest keyword volume in each dataset, and let w_1 vary with the number of update entries ranging from 10 to the highest keyword volume.

Fig. 6 and Fig. 7 illustrate search time costs rising linearly with the the number of update entries for varying values of w_2. We observe that FDXT outperforms SDSSE-CQ and presents a very close gap with ODXT. For instance, in Fig. 7(c), the overhead difference between FDXT and ODXT is about $0.01ms \sim 35.03ms$. This slight increase is due to the FDXT's need to generate trapdoors for cross-tag ciphertexts



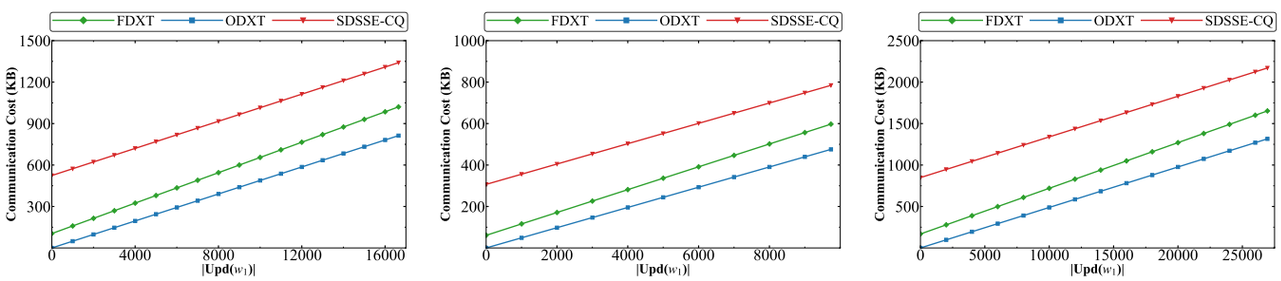
(a) Crime (b) Wikipedia (c) Enron

Fig. 9. Comparative analysis of the server's Search time overheads of FDXT, ODXT, and SDSSE-CQ with $|\mathbf{Upd}(w_2)|$ fixed to the highest keyword volume.



(a) Crime (b) Wikipedia (c) Enron

Fig. 10. Comparative analysis of communication costs of FDXT, ODXT, and SDSSE-CQ with a fixed $|\mathbf{Upd}(w_1)| = 10$.



(a) Crime (b) Wikipedia (c) Enron

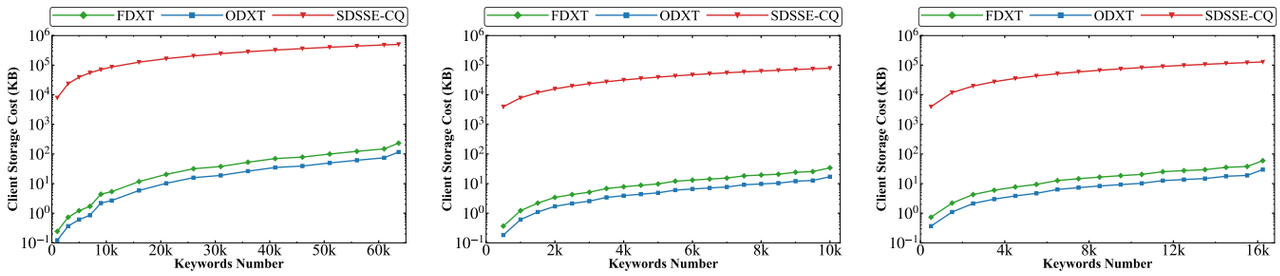
Fig. 11. Comparative analysis of communication overheads of FDXT, ODXT, and SDSSE-CQ with $|\mathbf{Upd}(w_2)|$ fixed to the highest keyword volume.

and ensure the forward privacy of **XSET**.

Fig. 8 and Fig. 9 present the search time costs for the client and server when the number of update entries of w_1 is increasing. We find that the time costs of FDXT and ODXT are of the same magnitude over all datasets. The time costs taken by both FDXT and ODXT increase proportionally with the *keyword volume* of w_1. Fig. 9(a) shows that the server's costs of FDXT and ODXT increase from 0.05ms and 0.04ms to 18.87ms and 15.17ms, respectively, with the maximum difference reaching only 3.70ms. This difference is primarily due to the decryption cost associated with the cross-tag ciphertexts.

Search communication costs. Fig. 10 and Fig. 11 present a comparison of the communication costs incurred by different schemes when processing the conjunctive search query $q = (w_1, w_2)$. We observe that the communication costs of both FDXT and SDSSE-CQ accumulate with the *keyword volume* of each queried keyword, and FDXT surpasses SDSSE-CQ. For instance, over the Wikipedia dataset, the average communication cost of FDXT is 180.08 KB, which is approximately 49.6% of that of SDSSE-CQ. Compared to ODXT, FDXT incurs a slight additional cost to achieve forward privacy. Across all three datasets, the maximum communication cost of FDXT remains as low as 1.61 MB.

Client storage costs. Fig. 12 shows a comparison of the client storage requirements across all datasets. Since SDSSE-CQ internally invokes two `Aura` instances, SDSSE-CQ needs to apply the bloom filters to store the deleted documents. As a result, the client storage costs of SDSSE-CQ are higher than those of FDXT and ODXT. The costs of FDXT closely align with those of ODXT, averaging only 1.02x higher according to Fig. 12(a), 12(b), and 12(c). This additional cost arises from the two extra counters introduced by FDXT for achieving



(a) Crime (b) Wikipedia (c) Enron

Fig. 12. Comparative analysis of the client storage costs of FDXT, ODXT, and SDSSE-CQ.

forward security. In practice, this cost remains acceptable.

To summarize, considering the overall costs of both Update and Search operations, FDXT outperforms SDSSE-CQ while achieving comparable performance to ODXT. In addition, since the cross-tag's ciphertexts in FDXT are independent of each other, FDXT can be further accelerated through multithreading techniques, achieving even better search performance. The above experiment results demonstrate that FDXT is a practical conjunctive DSSE scheme with both high efficiency and strong security.

VI. Conclusion

This paper investigates the forward privacy leakage and the keyword volume leakage in existing cross-tags-based conjunctive DSSE schemes. We analyze the causes of these leakages and formally capture them through stronger definitions of forward and backward privacy. We further present FDXT, a new conjunctive DSSE scheme satisfying these stronger securities. In short, FDXT redesigns cross-tags using encryption based on search and update counters to suppress the forward privacy and keyword volume leakages of cross-tags while reducing redundant ciphertext computations. Finally, we construct comprehensive experiments over three datasets to evaluate FDXT against existing schemes. The results show that FDXT achieves practical performance comparable to state-of-the-art conjunctive DSSE schemes in terms of search time, communication overhead, and storage cost.

References

[1] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *the ACM Conference on Computer and Communications Security, CCS'12, Raleigh, NC, USA, October 16-18, 2012* (T. Yu, G. Danezis, and V. D. Gligor, eds.), pp. 965–976, ACM, 2012.

[2] D. X. Song, D. A. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *2000 IEEE Symposium on Security and Privacy, Berkeley, California, USA, May 14-17, 2000*, pp. 44–55, IEEE Computer Society, 2000.

[3] MongoDB, "Aqueryable encryption." https://www.mongodb.com/docs/manual/core/queryable-encryption/, 2025. Accessed: Jun 16, 2025.

[4] CossackLabs, "Acra — data security solution for databases and distributed apps." https://www.cossacklabs.com/acra/, 2025. Accessed: Jun 16, 2025.

[5] IronCoreLabs, "Cloaked search." https://ironcorelabs.com/products/cloaked-search/, 2025. Accessed: Jun 16, 2025.

[6] E. Stefanov, C. Papamanthou, and E. Shi, "Practical dynamic searchable encryption with small leakage," in *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*, The Internet Society, 2014.

[7] Y. Zhang, J. Katz, and C. Papamanthou, "All your queries are belong to us: The power of file-injection attacks on searchable encryption," in *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10-12, 2016* (T. Holz and S. Savage, eds.), pp. 707–720, USENIX Association, 2016.

[8] X. Zhang, W. Wang, P. Xu, L. T. Yang, and K. Liang, "High recovery with fewer injections: Practical binary volumetric injection attacks against dynamic searchable encryption," in *32nd USENIX Security Symposium, USENIX Security 2023, Anaheim, CA, USA, August 9-11, 2023* (J. A. Calandrino and C. Troncoso, eds.), pp. 5953–5970, USENIX Association, 2023.

[9] R. Bost, B. Minaud, and O. Ohrimenko, "Forward and backward private searchable encryption from constrained cryptographic primitives," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017* (B. Thuraisingham, D. Evans, T. Malkin, and D. Xu, eds.), pp. 1465–1482, ACM, 2017.

[10] P. Xu, W. Susilo, W. Wang, T. Chen, Q. Wu, K. Liang, and H. Jin, "ROSE: robust searchable encryption with forward and backward security," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1115–1130, 2022.

[11] J. G. Chamani, D. Papadopoulos, M. Karbasforushan, and I. Demertzis, "Dynamic searchable encryption with optimal search in the presence of deletions," in *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022* (K. R. B. Butler and K. Thomas, eds.), pp. 2425–2442, USENIX Association, 2022.

[12] T. Chen, P. Xu, W. Wang, Y. Zheng, W. Susilo, and H. Jin, "Bestie: Very practical searchable encryption with forward and backward security," in *Computer Security - ESORICS 2021 - 26th European Symposium on Research in Computer Security, Darmstadt, Germany, October 4-8, 2021, Proceedings, Part II* (E. Bertino, H. Schulmann, and M. Waidner, eds.), vol. 12973 of *Lecture Notes in Computer Science*, pp. 3–23, Springer, 2021.

[13] T. Chen, P. Xu, S. Picek, B. Luo, W. Susilo, H. Jin, and K. Liang, "The power of bamboo: On the post-compromise security for searchable symmetric encryption," in *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*, The Internet Society, 2023.

[14] P. Mondal, J. G. Chamani, I. Demertzis, and D. Papadopoulos, "I/o-efficient dynamic searchable encryption meets forward & backward privacy," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024* (D. Balzarotti and W. Xu, eds.), USENIX Association, 2024.

[15] D. Cash, S. Jarecki, C. S. Jutla, H. Krawczyk, M. Rosu, and M. Steiner, "Highly-scalable searchable symmetric encryption with support for boolean queries," in *Advances in Cryptology - CRYPTO 2013 - 33rd Annual Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2013. Proceedings, Part I* (R. Canetti and J. A. Garay, eds.), vol. 8042 of *Lecture Notes in Computer Science*, pp. 353–373, Springer, 2013.

[16] S. Lai, S. Patranabis, A. Sakzad, J. K. Liu, D. Mukhopadhyay, R. Steinfeld, S. Sun, D. Liu, and C. Zuo, "Result pattern hiding searchable encryption for conjunctive queries," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018* (D. Lie, M. Mannan, M. Backes, and X. Wang, eds.), pp. 745–762, ACM, 2018.

[17] Q. Jiang, E. Chang, Y. Qi, S. Qi, P. Wu, and J. Wang, "Rphx: Result pattern hiding conjunctive query over private compressed index using intel SGX," *IEEE Trans. Inf. Forensics Secur.*, vol. 17, pp. 1053–1068, 2022.

[18] F. Li, J. Ma, Y. Miao, P. Wu, and X. Song, "Beyond volume pattern: Storage-efficient boolean searchable symmetric encryption with suppressed leakage," in *Computer Security - ESORICS 2023 - 28th European Symposium on Research in Computer Security, The Hague, The Netherlands, September 25-29, 2023, Proceedings, Part I* (G. Tsudik, M. Conti, K. Liang, and G. Smaragdakis, eds.), vol. 14344 of *Lecture Notes in Computer Science*, pp. 126–146, Springer, 2023.

[19] Y. Wang, S. Sun, J. Wang, X. Chen, J. K. Liu, and D. Gu, "Practical non-interactive encrypted conjunctive search with leakage suppression," in *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024* (B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, eds.), pp. 4658–4672, ACM, 2024.

[20] H. Liu, L. Xu, X. Liu, L. Mei, and C. Xu, "Query correlation attack against searchable symmetric encryption with supporting for conjunctive queries," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 1924–1936, 2025.

[21] D. Talapatra, S. Patranabis, and D. Mukhopadhyay, "Conjunctive searchable symmetric encryption from hard lattices," in *8th IEEE European Symposium on Security and Privacy, EuroS&P 2023, Delft, Netherlands, July 3-7, 2023*, pp. 958–978, IEEE, 2023.

[22] S. Kamara and T. Moataz, "Boolean searchable symmetric encryption with worst-case sub-linear complexity," in *Advances in Cryptology - EUROCRYPT 2017 - 36th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Paris, France, April 30 - May 4, 2017, Proceedings, Part III* (J. Coron and J. B. Nielsen, eds.), vol. 10212 of *Lecture Notes in Computer Science*, pp. 94–124, 2017.

[23] D. Yuan, C. Zuo, S. Cui, and G. Russello, "Result-pattern-hiding conjunctive searchable symmetric encryption with forward and backward privacy," *Proc. Priv. Enhancing Technol.*, vol. 2023, no. 2, pp. 40–58, 2023.

[24] S. Patranabis and D. Mukhopadhyay, "Forward and backward private conjunctive searchable symmetric encryption," in *28th Annual Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2021.

[25] A. Bag, S. Patranabis, and D. Mukhopadhyay, "Tokenised multi-client provisioning for dynamic searchable encryption with forward and backward privacy," in *Proceedings of the 19th ACM Asia Conference on Computer and Communications Security, ASIA CCS 2024, Singapore, July 1-5, 2024* (J. Zhou, T. Q. S. Quek, D. Gao, and A. A. Cárdenas, eds.), ACM, 2024.

[26] C. Guo, W. Li, X. Tang, K. R. Choo, and Y. Liu, "Forward private verifiable dynamic searchable symmetric encryption with efficient conjunctive query," *IEEE Trans. Dependable Secur. Comput.*, vol. 21, no. 2, pp. 746–763, 2024.

[27] L. Chen, J. Li, and J. Li, "Toward forward and backward private dynamic searchable symmetric encryption supporting data deduplication and conjunctive queries," *IEEE Internet Things J.*, vol. 10, no. 19, pp. 17408–17423, 2023.

[28] C. Zuo, S. Lai, S. Sun, X. Yuan, J. K. Liu, J. Shao, H. Wang, L. Zhu, and S. Cui, "Searchable encryption for conjunctive queries with extended forward and backward privacy," *Proc. Priv. Enhancing Technol.*, vol. 2025, no. 1, pp. 440–455, 2025.

[29] J. G. Chamani, D. Papadopoulos, C. Papamanthou, and R. Jalili, "New constructions for forward and backward private symmetric searchable encryption," in *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, CCS 2018, Toronto, ON, Canada, October 15-19, 2018* (D. Lie, M. Mannan, M. Backes, and X. Wang, eds.), pp. 1038–1055, ACM, 2018.

[30] S. Sun, R. Steinfeld, S. Lai, X. Yuan, A. Sakzad, J. K. Liu, S. Nepal, and D. Gu, "Practical non-interactive searchable encryption with forward and backward privacy," in *28th Annual Network and Distributed System Security Symposium (NDSS)*, The Internet Society, 2021.

[31] L. Blackstone, S. Kamara, and T. Moataz, "Revisiting leakage abuse attacks," in *27th Annual Network and Distributed System Security Symposium, NDSS 2020, San Diego, California, USA, February 23-26, 2020*, The Internet Society, 2020.

[32] S. Oya and F. Kerschbaum, "IHOP: improved statistical query recovery against searchable symmetric encryption through quadratic optimization," in *31st USENIX Security Symposium, USENIX Security 2022, Boston, MA, USA, August 10-12, 2022* (K. R. B. Butler and K. Thomas, eds.), pp. 2407–2424, USENIX Association, 2022.

[33] M. Damie, F. Hahn, and A. Peter, "A highly accurate query-recovery attack against searchable encryption using non-indexed documents," in *30th USENIX Security Symposium, USENIX Security 2021, August 11-13, 2021* (M. D. Bailey and R. Greenstadt, eds.), pp. 143–160, USENIX Association, 2021.

[34] H. Nie, W. Wang, P. Xu, X. Zhang, L. T. Yang, and K. Liang, "Query recovery from easy to hard: Jigsaw attack against SSE," in *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024* (D. Balzarotti and W. Xu, eds.), USENIX Association, 2024.

[35] J. Wu, K. Zhang, L. Wei, J. Gong, and J. Ning, "Practical searchable symmetric encryption for arbitrary boolean query-join in cloud storage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 10086–10098, 2024.

[36] J. Li, J. Wang, R. Zhang, Y. Xin, and W. Xu, "NEMO: practical distributed boolean queries with minimal leakage," *IEEE Trans. Inf. Forensics Secur.*, vol. 19, pp. 2594–2608, 2024.

[37] C. S. Jutla and S. Patranabis, "Efficient searchable symmetric encryption for join queries," in *Advances in Cryptology - ASIACRYPT 2022 - 28th International Conference on the Theory and Application of Cryptology and Information Security, Taipei, Taiwan, December 5-9, 2022, Proceedings, Part III* (S. Agrawal and D. Lin, eds.), vol. 13793 of *Lecture Notes in Computer Science*, pp. 304–333, Springer, 2022.

[38] D. Cash, J. Jaeger, S. Jarecki, C. S. Jutla, H. Krawczyk, M. Rosu, and M. Steiner, "Dynamic searchable encryption in very-large databases: Data structures and implementation," in *21st Annual Network and Distributed System Security Symposium, NDSS 2014, San Diego, California, USA, February 23-26, 2014*, The Internet Society, 2014.

[39] R. Bost, "$\sum o\varphi o\varsigma$: Forward secure searchable encryption," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security, Vienna, Austria, October 24-28, 2016* (E. R. Weippl, S. Katzenbeisser, C. Kruegel, A. C. Myers, and S. Halevi, eds.), pp. 1143–1154, ACM, 2016.

[40] D. Liu, W. Wang, P. Xu, L. T. Yang, B. Luo, and K. Liang, "d-dse: Distinct dynamic searchable encryption resisting volume leakage in encrypted databases," in *33rd USENIX Security Symposium*, USENIX Association, 2024.

[41] C. D. Portal, "Chicago crimes reports." https://data.cityofchicago.org/, 2025. Accessed: Mar 26, 2025.

[42] W. Foundation, "Wikimedia downloads." https://dumps.wikimedia.org/, 2025. Accessed: Mar 26, 2025.

[43] G. Attardi, "Wikiextractor." https://github.com/attardi/wikiextractor, 2021. Accessed: Mar 26, 2025.

[44] K. S. Kim, M. Kim, D. Lee, J. H. Park, and W. Kim, "Forward secure dynamic searchable symmetric encryption with efficient updates," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017* (B. Thuraisingham, D. Evans, T. Malkin, and D. Xu, eds.), pp. 1449–1463, ACM, 2017.

[45] C. Project, "Enron email dataset." https://www.cs.cmu.edu/~enron/, 2015. Accessed: Mar 26, 2025.

[46] M. F. Porter, "An algorithm for suffix stripping," *Program*, vol. 14, no. 3, pp. 130–137, 1980.